

ch 3 and ch 4

- 1) complete all the questions about class diagram and communication/collaboration diagram from regular Old Question 2077, 2078, 2079, 2080, 2081

2081 chaira

Qno 4

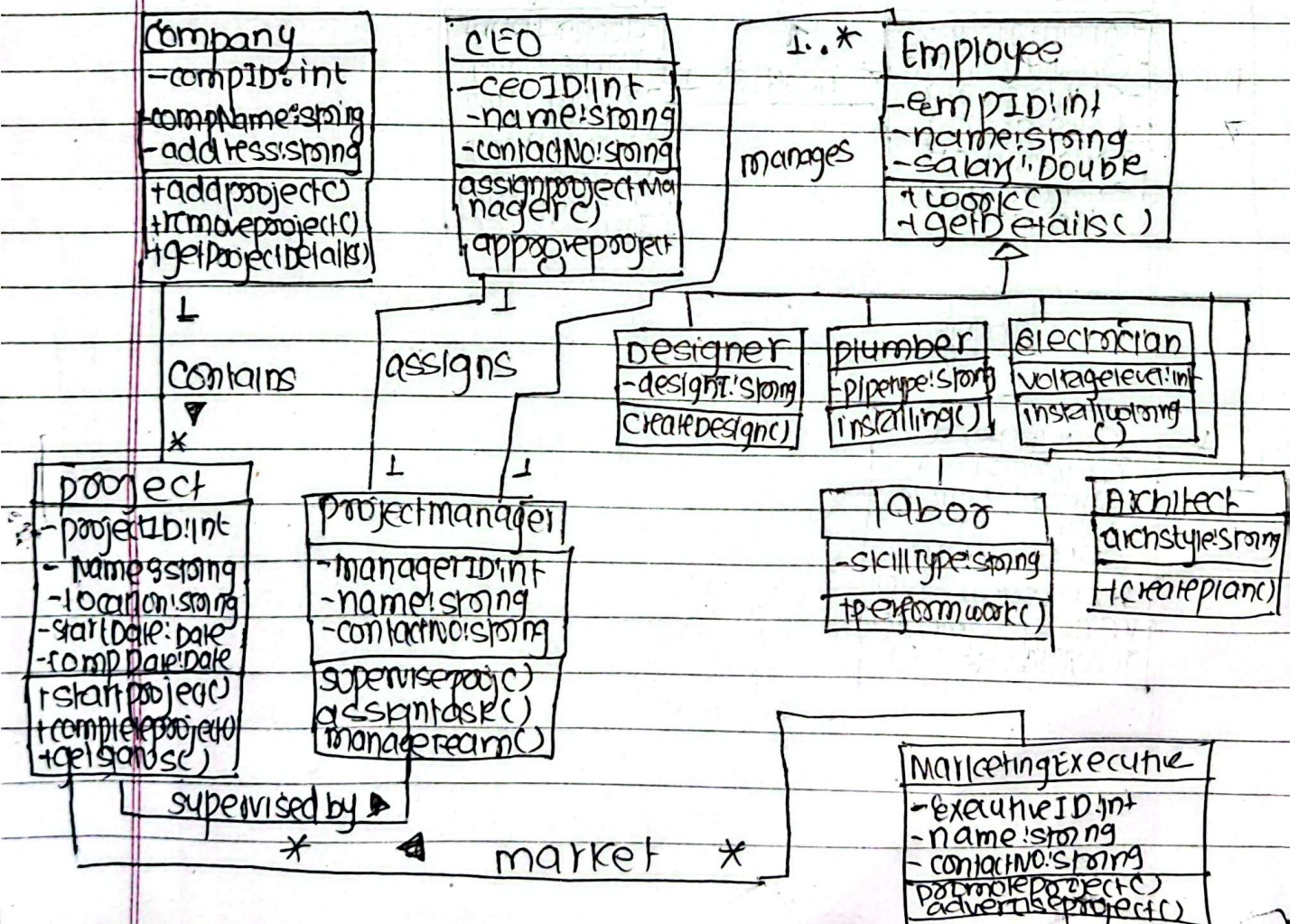
Class % company: companyID, companyName, address
CEO: CEOID, name, contactNo

project: projectID, projectName, location, startDate, completionDate

projectManager: ManagerID, name, contactNo

Employee: EmpID, name, salary
designer, plumber, electrician, architect,
labor

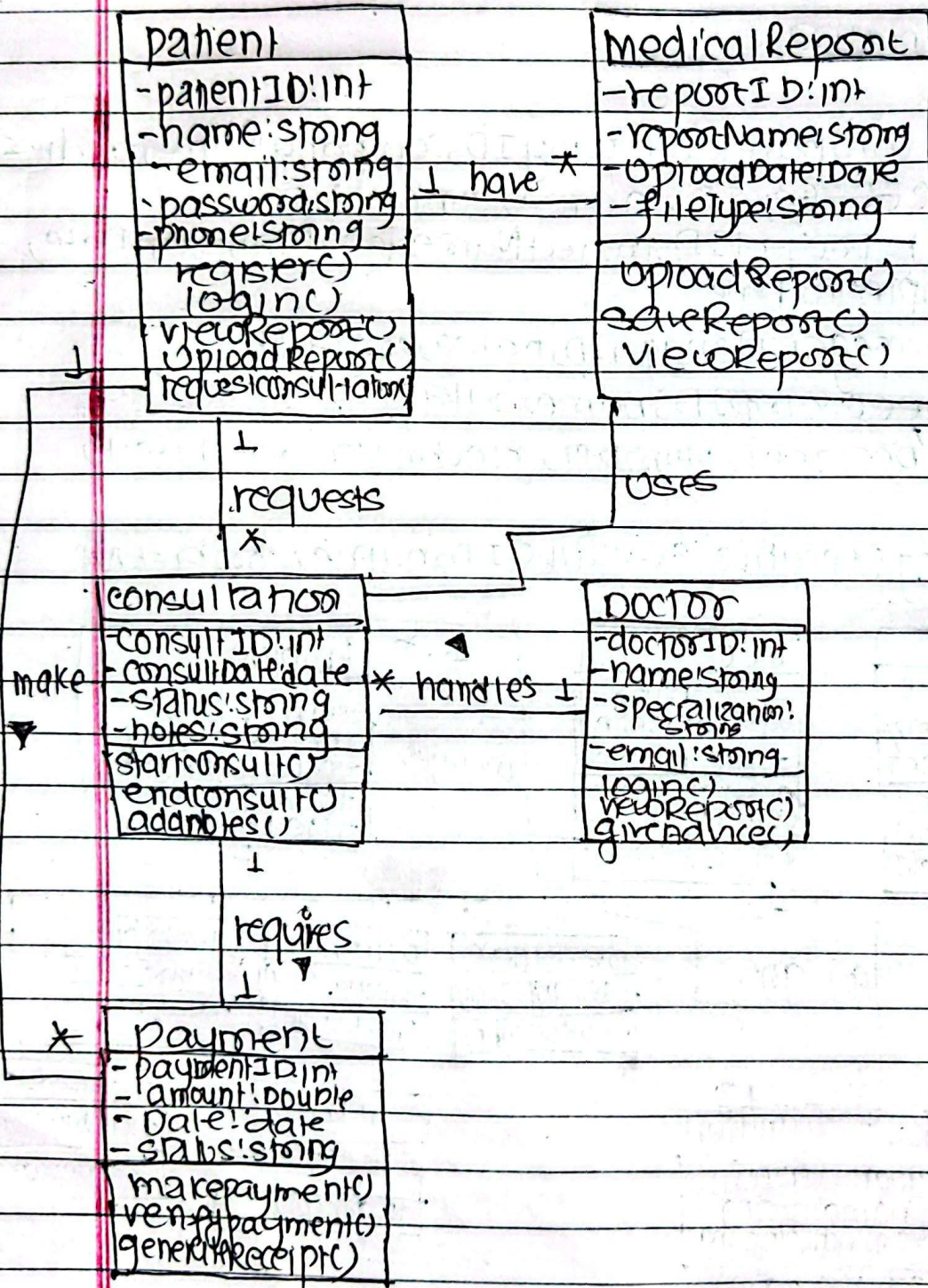
Marketing Executive: Executive ID, name, contactNo



2080 chaitra

Gno: G

class diagram for online doctor consultation.



2079 chaitoz

@no9)

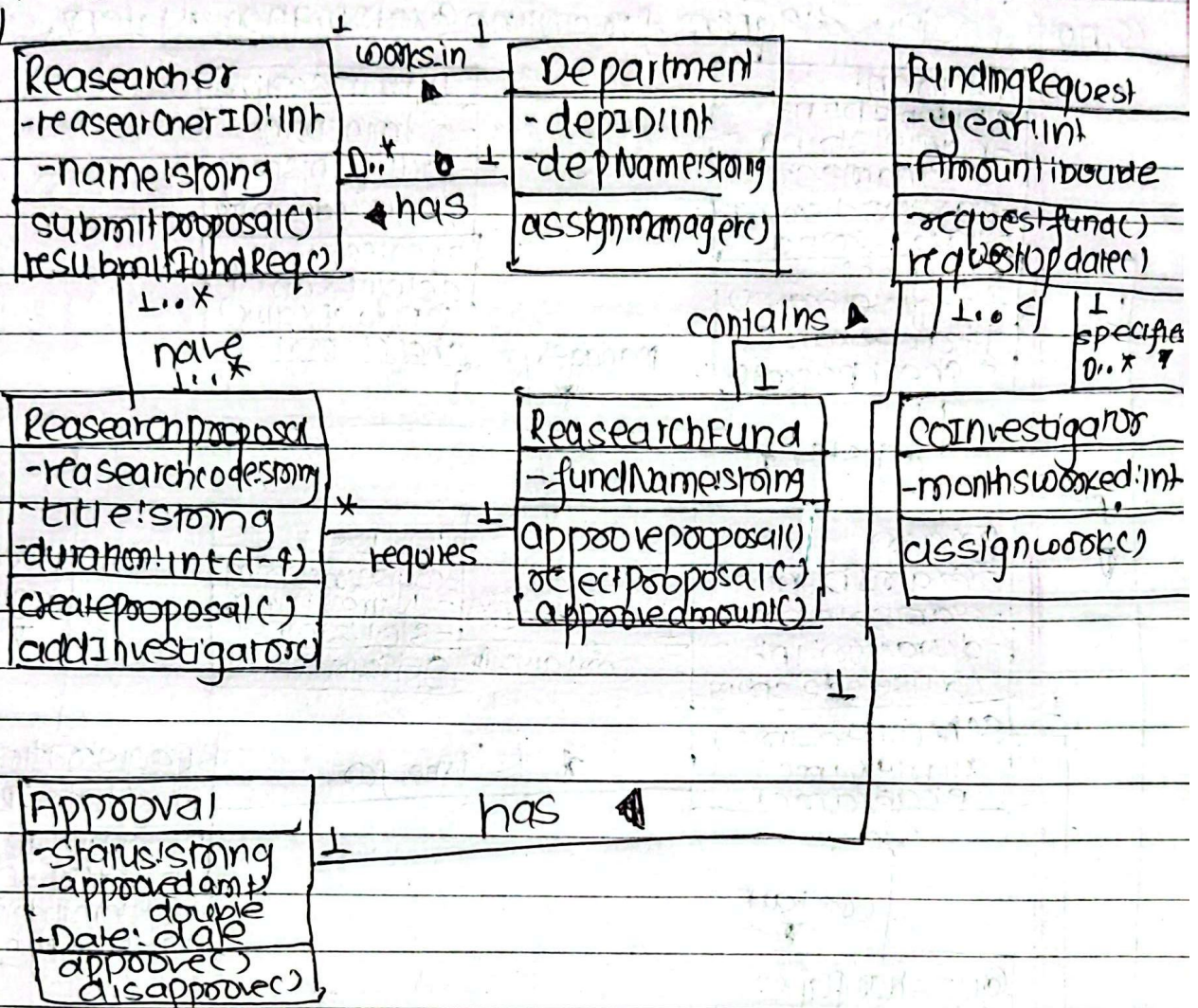
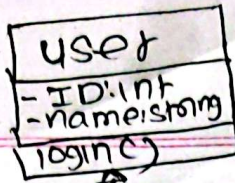
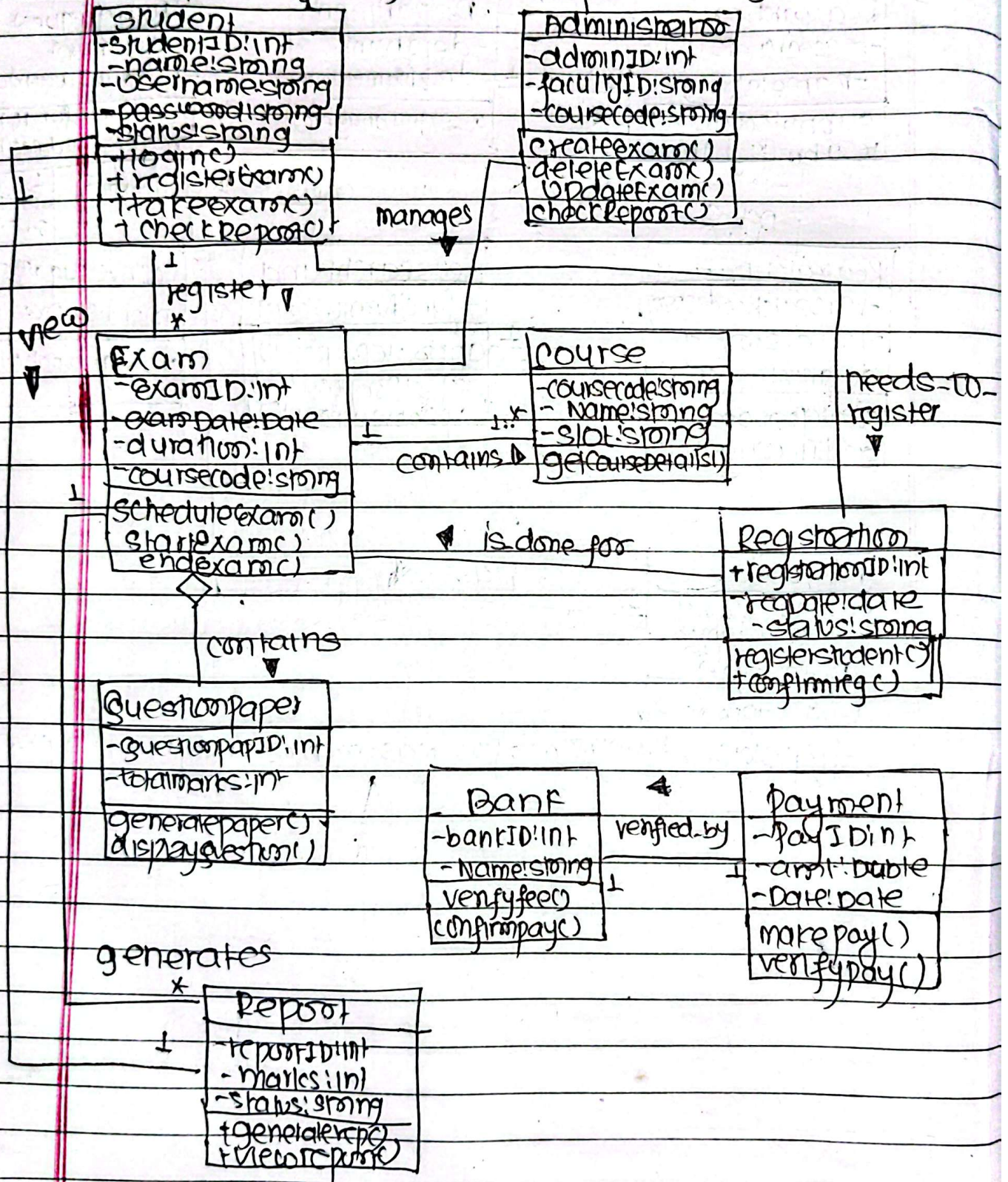


fig: class diagram for research management system



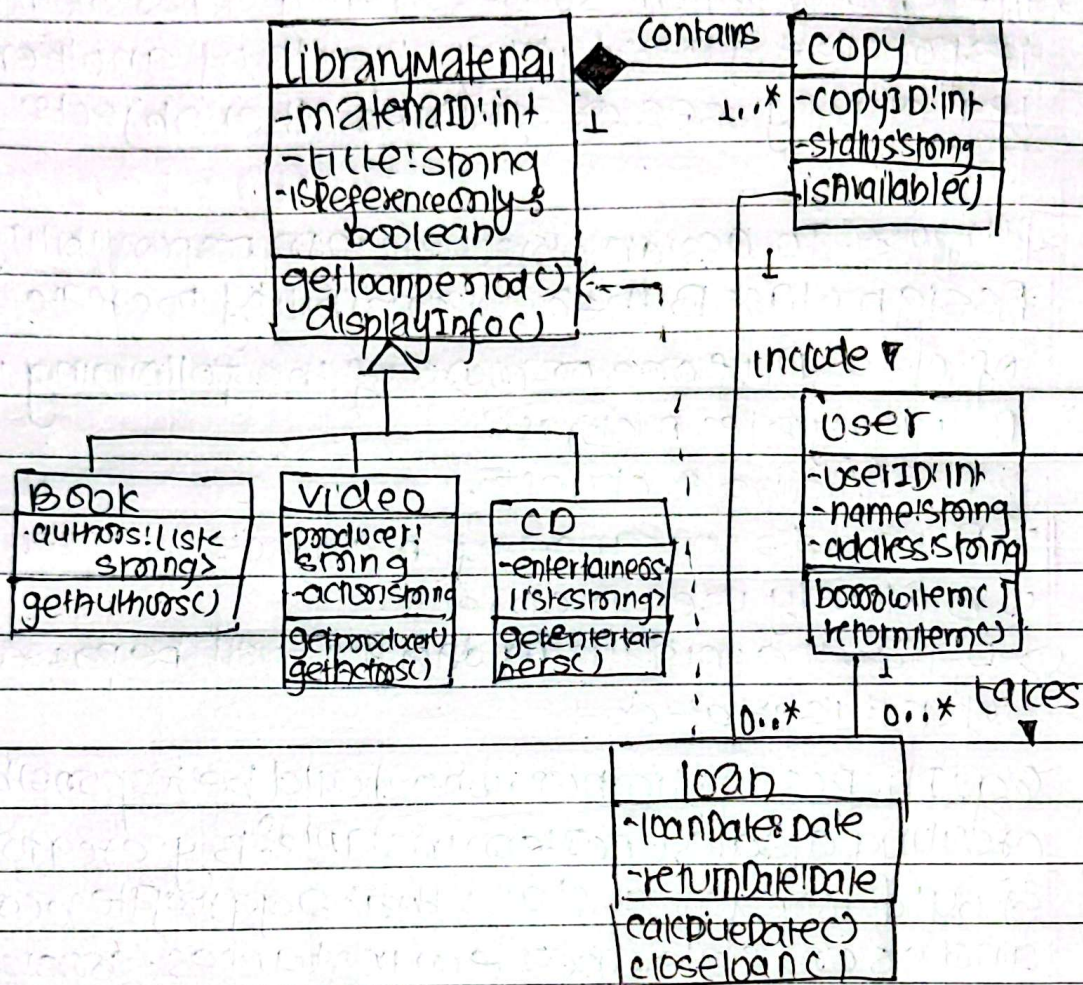
2078 chaitra

Q no 6 class diagram for online examination system.



2077 chaito

Qno.5 & class diagram for library system.



- 2) Describe the term design pattern. Explain the purpose & benefits of information expert, creator, controller & polymorphism defined in GRASP.
- A design pattern is a written document that describes a general solution to design problem that reoccurs repeatedly in many projects. It contains elements like pattern name, problem description, solution description, consequences.
- ↳ GRASP describe fundamental principle of object design & responsibility assignment, expressed as pattern.

The purpose and benefit are :

i) creator

→ The creator pattern suggests that a class should be responsible for creating objects of another class if it closely uses or contains other objects.

Purpose: To assign object creation responsibility logically. Assign class B the responsibility to create an instance of class A if one or more of the following is true:

→ B aggregates A objects

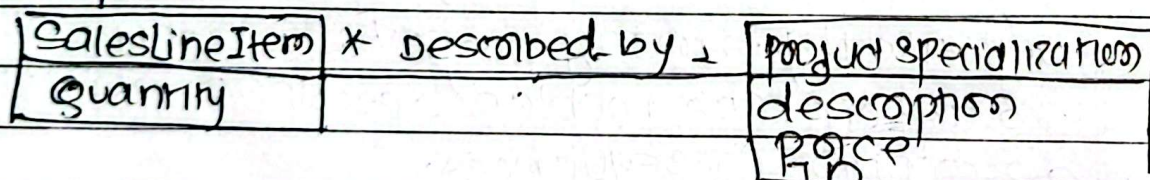
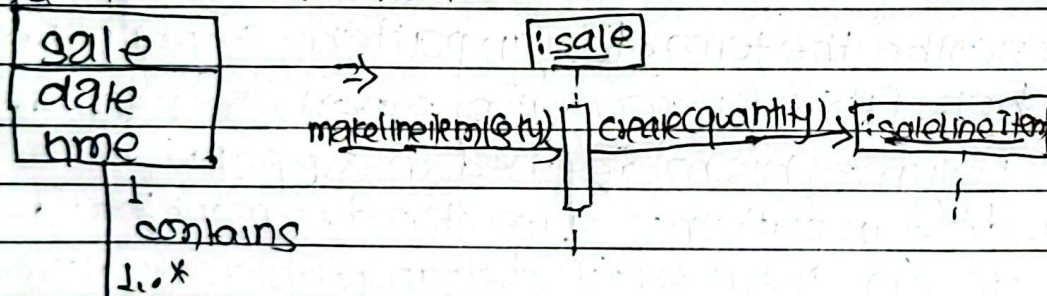
→ B contains A objects.

→ B records instance of A objects.

→ B closely uses A objects.

→ B has the initializing data that will be passed to A when it is created.

eg: In pos application, who should be responsible for creating a saleslineItem instance? By creator, we should look for a class that aggregates, contains, and so on, saleslineItem instances. Consider the partial domain model below.



Since sale contains (in fact, aggregates) many saleslineItem object, the creator pattern suggests that sale is a good candidate to have responsibility of creating saleslineItem instances.

Benefit

- ↳ promotes low coupling
- ↳ Improves cohesion
- ↳ makes object creation clear & logical

ii) Information expert

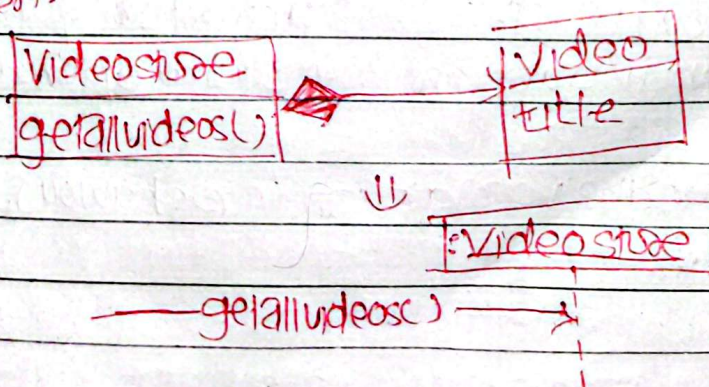
- ↳ The information expert pattern states that ~~responsibility~~ ^{responsibility} should be assigned to the class that has the necessary information to fulfill it.

purpose: To place responsibilities in classes that have the required data, which leads to better encapsulation & lower coupling

Benefit

- ↳ Improves encapsulation
- ↳ Reduces unnecessary communication between classes
- ↳ Makes the system easier to maintain & understand

Eg: Assume we need to get all the videos of a video store. Since video store knows about all the videos, we can assign this responsibility of given all videos to Video store class. so video store is the information expert.



c) Controller

→ A controller pattern assigns responsibility to a class that handles system events & coordinates action between different objects.

Purpose: To separate user interface from business logic.

Benefit

- ↳ Improves system organization
- ↳ Enhances maintainability.
- ↳ Reduces coupling between UI & business objects.

eg: In an online examination system:

```

graph LR
    Student[Student] --> submits[submits Exam]
    submits --> Controller["Exam Controller -> process Exam()"]
    Controller --> Exam["Exam -> CalculateResult()"]
    
```

The controller receives the request and delegates work to appropriate classes.

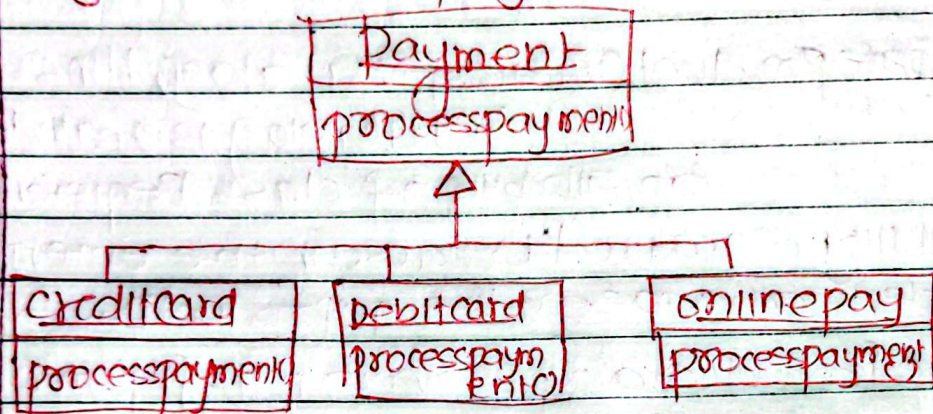
d) Polymorphism

→ Polymorphism means different classes can respond to the same method in different ways. It is used when different objects require different behaviour for the same operations.

Purpose: To handle variation in behaviour based on object type.

benefits

- Eliminates complex conditional statements
 - Improves flexibility & extensibility
 - Makes the system easier to extend with new types
- eg! consider a payment system



Here, each class implements `processpayment()`.
But the implementation is different for each payment type

- 2) What is visibility? Explain its types with example.
- Visibility is the ability of an object to "see" or have a reference to another object.

There are four common ways that visibility can be achieved from object A to object B.

- 1) Attribute visibility.
Attribute visibility from A to B exists when B is attribute of A. It is relatively permanent visibility because it persists as long as A & B exist. This is a very common form of visibility in object-oriented systems.

→ To illustrate, in a java class definition for Register, a Register instance may have attribute visibility to a ProductCatalog, since it is an attribute of the Register.

```
public class Register
```

```
{
    private ProductCatalog catalog; // instance of
    // ProductCatalog is created but as
    // an attribute of class Register.
}
```

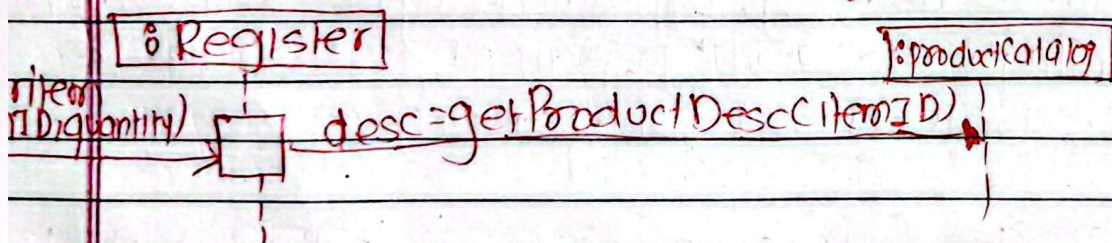
This visibility is required because in the enterItem, a Register needs to send the getProductDescription message ProductCatalog as shown in figure below:-

```
class Register
```

```
public void enterItem(itemID)
{
    // ...
}
```

```
{
    private ProductCatalog catalog;
}
```

```
    desc = catalog.getProductDe
    // ...
    desc(itemID);
}
```

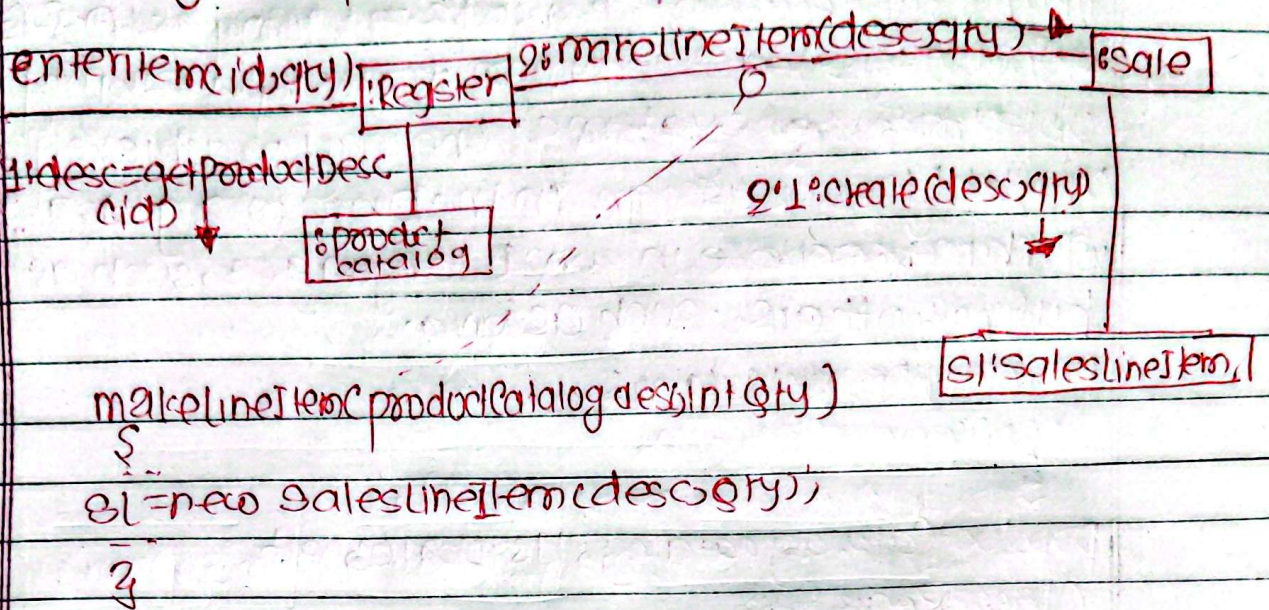


b) parameter visibility

→ parameter visibility from A to B exists when B is passed as a parameter to a method of A. It is a relatively temporary visibility because it persists only within the scope of the method.

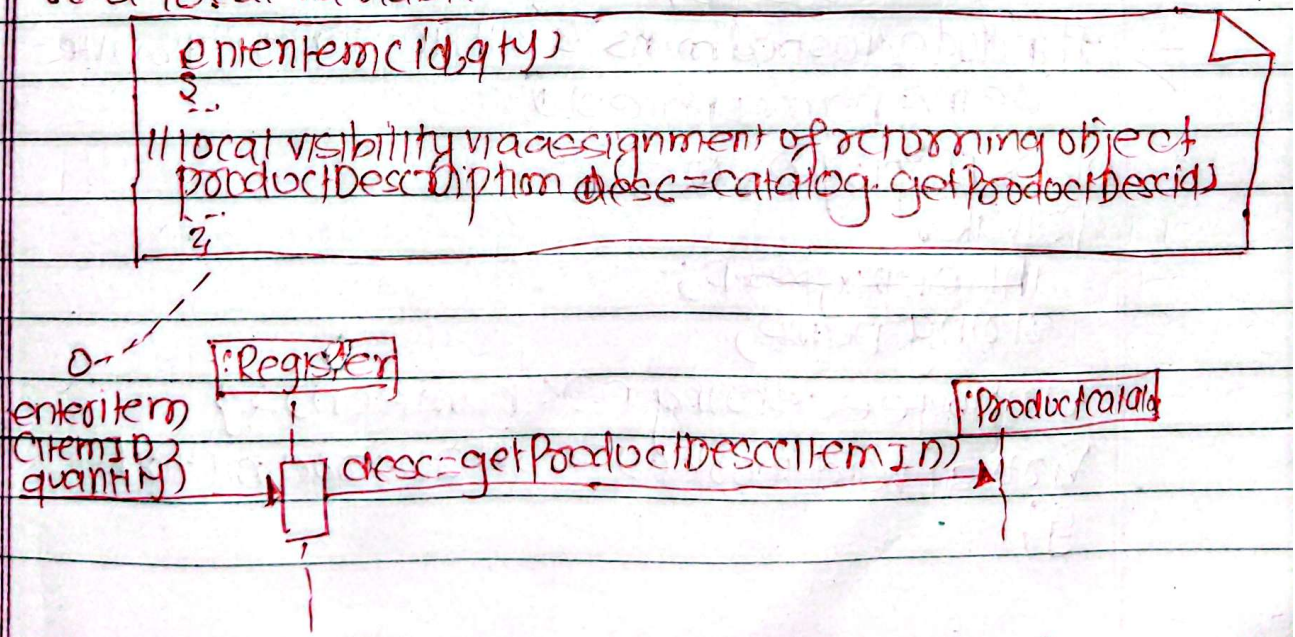
→ To illustrate, when the makeLineItem message is sent to a sale instance a ProductDescription instance is passed as a parameter. Within the scope of the

makeLineItem method, the sale has parameter visibility to a productDescription.



iii) local visibility :- local visibility from A to B exists when B is declared as local object within a method of A. It is a relatively temporary visibility because it persists only within the scope of the method. Two common means by which local visibility is achieved are:-

- create a new local instance & assign it to a local variable
- Assign the returning object from a method invocation to a local variable



d) Global visibility: Global visibility from A to B exists when B is global to A. It is a relatively permanent visibility because it persists as long as A & B exist. It is least common form of visibility in object-oriented systems. One way to achieve global visibility is to assign an instance to a global, which is possible in some languages, such as C++, but not others, such as Java.

Eg. class ProductCatalog {

{

ProductCatalog catalog; // global object

Class Registers

public:

void enterItem(int itemID);

Catalog.getProductDes(itemID); // global visibility

{

};

4) Explain the process of mapping design to code with reference to class diagram & interaction diagram drawn from 2081. Also implement the design of 2080 class diagram.

→ #include <iostream.h> // with lesser attributes using namespace std;

class Employee {

private:

int employeeID;

string name;

Employee(string n): name(n) {}

virtual void work() {} // abstract method

};

```
class Designer: public Employee {
public:
    Designer(string n): Employee(n) {}
    void work()
    {
        cout << name << "is designing.\n";
    }
}
```

```
class plumber: public employee {
public:
    plumber(string n): Employee(n) {}
    void work()
    {
        cout << name << "is plumbing.\n";
    }
}
```

```
class projectManager {
public:
    string name;
    projectManager(string n, name(n)) {}
    void assignTask(Employee &e, string task) {
        cout << "Assigning" << task << "to" << e.name << endl;
    }
}
```

```
class project {
public:
    string name;
    projectManager *manager;
    project(string n, projectManager *m): name(n),
        manager(m) {}
    void start() { cout << name << "started.\n"; }
    void complete() {}
}
```

```
class CEO {
public:
```

```

string name;
CEO (string n): name(n) {}
void assignManager (project &p, projectManager
&m) {
    p.manager = &m;
    cout << "CEO" << name << "assigned manager" <<
    m.name << endl;
}
};

int main () {
    CEO ceo ("Alice");
    projectManager pm ("Bob");
    project proj ("Bridge construction", &pm);
    ceo.assignManager (proj, pm);
    Designer d ("Charlie");
    Plumber p ("David");
    pm.assignTask (d, "Design");
    pm.assignTask (p, "Install pipes");
    d.work();
    p.work();
    proj.start();
    proj.complete();
    return 0;
}

```

(Q5) Q. How can we create definition and methods from the design class diagram & interaction diagram. Explain with appropriate example.

7 In object-oriented design, Design class diagram (DCD) & interaction diagrams help in identifying the methods that must be implemented in the classes. These diagrams describe the structure & behaviour of a system, which can be translated into method definitions in programming.

A design class diagram shows the static structure of a system. It contains classes, attributes (data member), methods (operations), Relationship between classes.

Steps to create methods from DCD are:

- Identify the classes in the diagram.
- Extract the attribute of each class which represent the data stored in the class.
- Identify the methods listed in the class.
- Determine the visibility of attributes & methods such as (+) public, (-) private, (#) protected.
- convert these operations into method definitions in code.

Thus, the operations in class diagram directly becomes methods in the implementation.

An interaction diagram (eg, a sequenced diagram) represent the dynamic behaviour of the system by showing how objects communicate with each other through message. Each message sent from one object to another represents a method call.

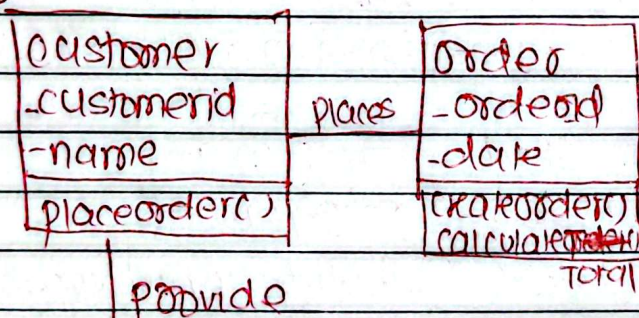
Steps to create methods from interaction diagram:

- 1) Identify the objects involved in the interaction
- 2) Observe the message exchanged between objects.
- 3) The object receiving the message must implement the corresponding method.

4) Determine the parameters & our values based on the message information.

Therefore each message in the interaction diagram becomes a method in receiving class.

eg: online order system



Interaction scenario

- 1) customer sends placeorder() message
- 2) customer request order to createorder()
- 3) order initially call calculateorder() to compute total price
- 4) order sends a message to payment to processpayment()

```
class customer
{
    private:
    int customerID;
    string name;
    public void placeorder()
    {
        order o = new order();
        o.createorder();
    }
}
```

```
class order
{
    int orderID;
    string date;
    public void createorder()
    {
        calculateTotal();
    }

    public double calculateTotal()
    {
        // logic for calculating total
        return 0;
    }
}
```

```
class payment
{
    int paymentID;
    double amount;
    public boolean processpayment(double amount)
    {
        return true;
    }
}
```

Q6) Explain the concept of ~~interface~~ & implementation in OOD & implementation.

→ An interface defines the set of operations or services that a class provides to other classes. It specifies method names, parameters, & return types, but it doesn't include the internal logic of how methods work. Thus, the interface represents the external view of the class.

Characteristics

- ↳ Describe what a class can do
- ↳ contains method declaration only
- ↳ doesn't include method bodies or internal logic
- ↳ Helps in achieving abstraction & loose coupling.

eg:

interface payment

{

void processPayment(double amount)

}

In this example, interface defines a method ProcessPayment() but doesn't specify how the payment will process.

Implementation refers to the actual coding of the methods defined in the interface. It contains internal logic & algorithms that perform the required operations. Thus, implementation represents how the functionality is carried out.

Characteristics

- ↳ contains the method definitions

- ↳ provides the actual functionality of the program.
- ↳ can vary w/o affecting other classes that use the interface.
- ↳ Helps in interface hiding.

eg:

```
class onlinepayment implements payment
```

```
{
```

```
    public void processpayment(double amount)
```

```
{
```

```
        System.out.println("processing online payment of  
        + amount);
```

```
    }
```

Here the class onlinepayment implements the interface payment & provides the actual logic for processing the payment

Q.7 Write short notes on!

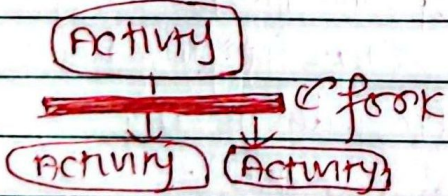
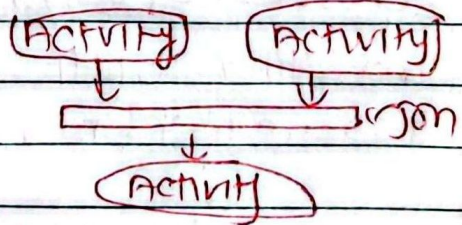
1) Dewey Decimal Numbering & its use in modeling

→ Dewey Decimal Numbering is a hierarchical technique used to organize & structure complex systems in s/w modeling. It is derived from the Dewey Decimal system used in libraries for classifying books. In modeling, it helps in breaking down large systems into smaller & manageable components.

In this method elements are numbered in a hierarchical manner such as 1.1.1.1.1.2, 2.2.1.5 etc. Each level represents a deeper level of detail or decomposition. For eg, a major process may be labeled as 1, and its

Sub-processes as 1.1.1, 1.1.2 & further detailed activities as 1.1.1.1, 1.1.1.2. This numbering system is commonly used in use case models, activity diagram to maintain clarity & organization improving readability & systematic representation of complex models

ii) Forking vs Joining

Basis	Forking	Joining
Define	Forking is the process of splitting one control flow into multiple parallel flows in an activity diagram.	Joining is the process of combining multiple parallel flows into a single flow.
Purpose	It allows several activities to start & run simultaneously	It synchronizes parallel activities & continues the process only after all activities are completed
Flow direction	One incoming flow and multiple outgoing flows	Multiple incoming flows & one outgoing flow
Execution	Activities execute concurrently after the fork.	Activities must finish before continuing after the join
Representation	 <pre> graph TD A[Activity] --> Fork[] Fork --> B[Activity] Fork --> C[Activity] </pre>	 <pre> graph TD A[Activity] --> Join[] B[Activity] --> Join Join --> C[Activity] </pre>

iii) Conceptual model vs Implementation model

Basis	Conceptual model	Implementation model
Definition	A conceptual model represents the system from high-level & real-world perspective.	A implementation model represent how the system will be built & implemented in s/w.
Focus	Focuses on understanding the problem domain & system concepts.	Focuses on technical details such as classes, algorithms, & data structures.
Dependence	Independent of programming languages & technologies.	Dependent on programming languages, frameworks & platforms.
Purpose	Used during system analysis & requirement understanding.	Used during system design & coding.
Example	Identifying entities like customer, order, product & their relationships.	Defining classes, methods, DB tables, & program structure for those entities.

e) Model view controller (MVC)

→ MVC stands for model view controller, which is a widely used architectural pattern in s/w development, especially in web applications. It separates the application into three interconnected components to improve modularity and maintainability.

The model represents the data and business logic of the application. It manages the system's data rules, & operations such as storing, retrieving & updating information.

The view represents the user interface. It displays the data from the model to the user in visual form such as web pages, forms or graphical interface.

The controller acts as an intermediary between the model & the view. It receives user i/p, processes it, & updates the model or view accordingly.

The MVC architecture improves separation of concerns, allowing independent development of components, & make the system easier to maintain & scale.

e) Agile method

Agile is a modern software development methodology that emphasizes flexibility, collaboration, & iterative progress. Instead of developing the entire system at once, Agile divides the project into small cycles called iterations or sprints, each producing a working version of the software. Each sprint includes:

- a) planning
- b) Design
- c) Development
- d) Testing
- e) Review

A working product is delivered at the end of every sprint.

Advantage

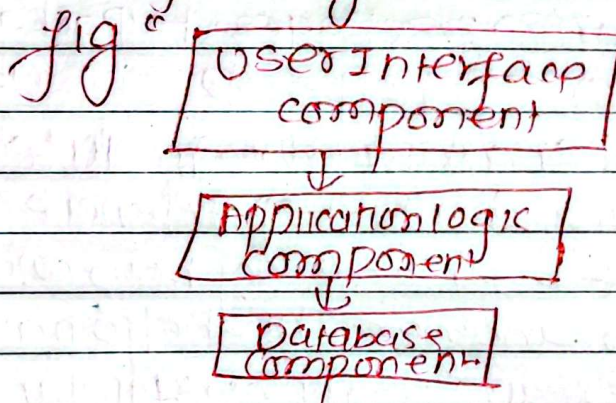
- ↳ Highly flexible & adaptable to changes
- ↳ Working software is delivered frequently
- ↳ Reduces time to market.

Disadvantages

- ↳ Requires experience & disciplined team
- ↳ Less focus on documentation

f) Component Diagram

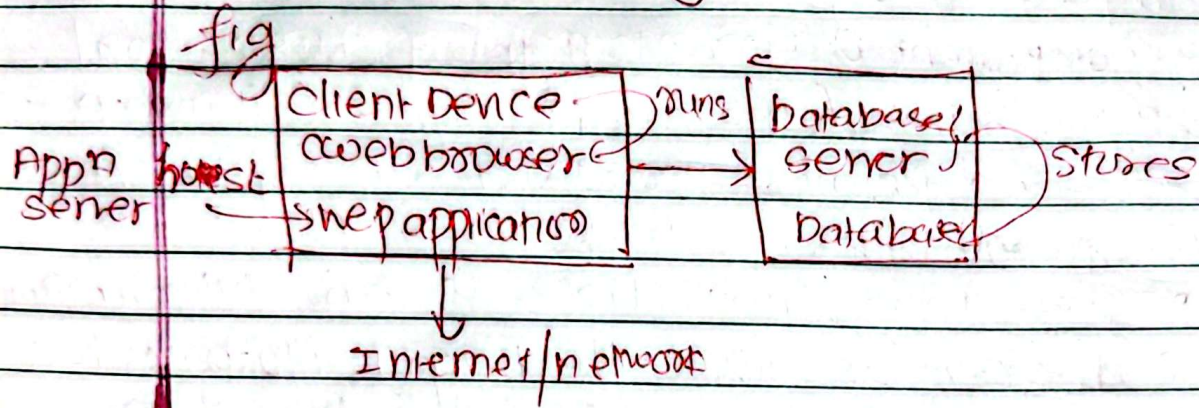
A component diagram is a type of UML structural diagram that shows the organization & relationship between different s/w components in a system. A component represents a modular & replaceable part of the system, such as a s/w module, library, subsystem, or service. The diagram illustrates how components are connected & how they depend on each other through interface & dependencies. It helps developers understand the high-level architecture of the system & how different modules interact.



g) Deployment Diagram

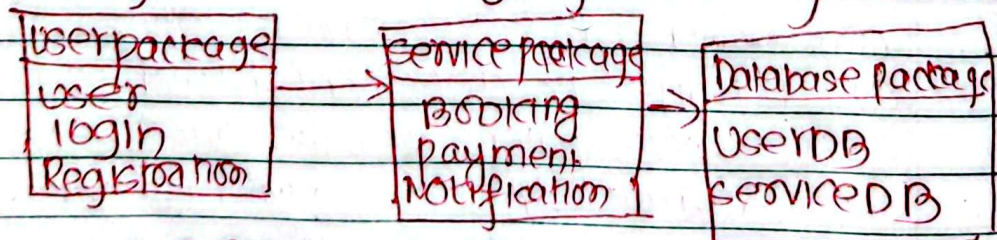
- ↳ Deployment Diagram is a UML diagram used to represent the physical arrangement of h/w & s/w components in a system. It shows how s/w artifacts are deployed on different h/w nodes such as servers, computers, mobile devices, or network components. In a deployment diagram, nodes represent physical device or execution environments, while artifacts represent s/w components deployed on those

nodes. The diagram also shows the communication lines between the nodes. The diagram is mainly used to model the runtime architecture of the system, including n/w configuration & system architecture.



n) Package Diagram

A package diagram is used in UML to organize & group related elements such as classes, interfaces and components into packages. A package acts as a container that helps structure a large system into smaller, manageable modules. Package diagram shows dependencies between packages, indicating how one package relies on another. This helps maintain a clear structure & reduces complexity in large projects. By organizing system elements into packages, developers can improve modularity, maintainability, & readability of the design.



Classes are grouped into package.
Dependencies shows how one package uses another packages.

i) CRC cards

ii) CRC stands for class Responsibility collaboration.

Although not formally a part of UML, CRC cards helps to assign responsibilities & indicate collaboration with other objects. CRC cards are index cards, one for each class, upon which the responsibilities of class are briefly written, & a list of collaborator objects to fulfill those responsibilities. Each card contains:

a) class

b) Responsibilities that the class knows or does.

c) collaborators (other classes that the class interacts with to complete its responsibilities).

CRC layout →

class name	
Responsibilities	collaboration

CRC examples

customer	
withdraw cash	Account
deposit cash	Branch
check balance	

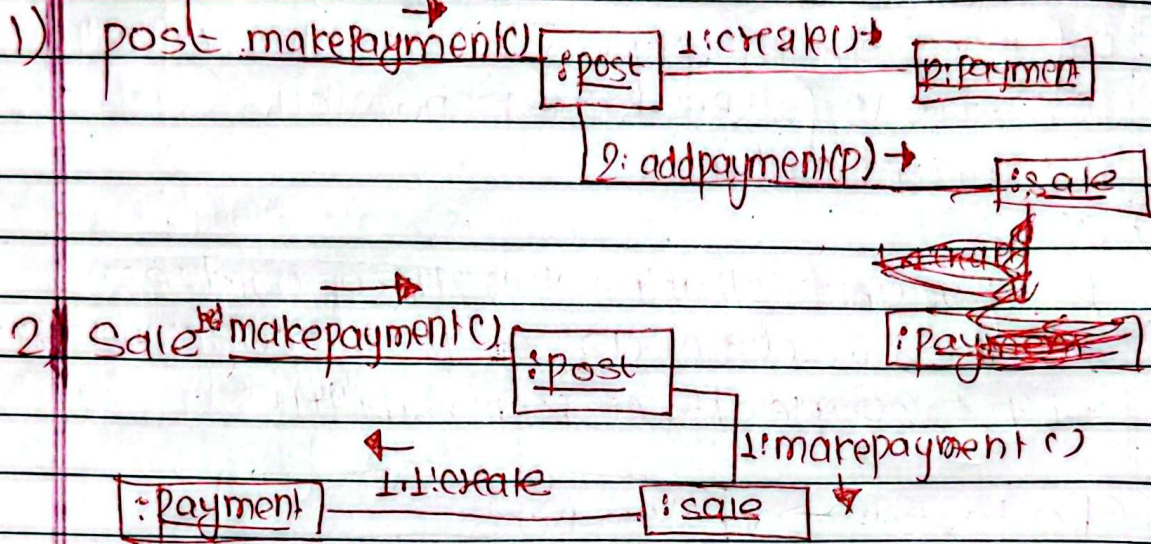
i) low coupling & high cohesion.

→ low coupling refers to the degree of dependency between classes. Low coupling assign a responsibility so that coupling remains low. When coupling is low, classes are loosely connected, meaning changes in one class have minimal impact on others. This improves system flexibility & maintainability.

eg: Assume we need to create payment instances & associate it with the sale. what class should be responsible for this?

Payment Post Sale

Two possibilities



In 1, Register creates the payment & adds coupling of register to payment & sale. An alternative solution to creating the payment and associating it with the sale.

Sale does the creation of payment, doesn't increase the coupling. purely from the point of view of coupling. Design 2 is preferable because overall low coupling is maintained.

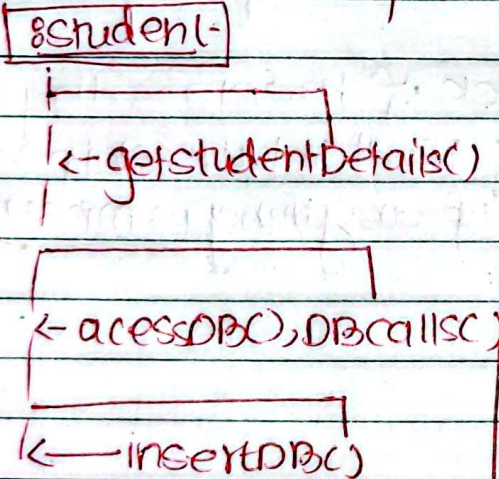
High cohesion refers to how closely related the responsibilities of a class are. A highly cohesive class focuses on a single well-defined task or purpose, making it easier to understand & maintain. with high cohesion:

- ↳ classes are easier to maintain
- ↳ easier to understand
- ↳ often supports low coupling
- ↳ support arose because of fine grained responsibility

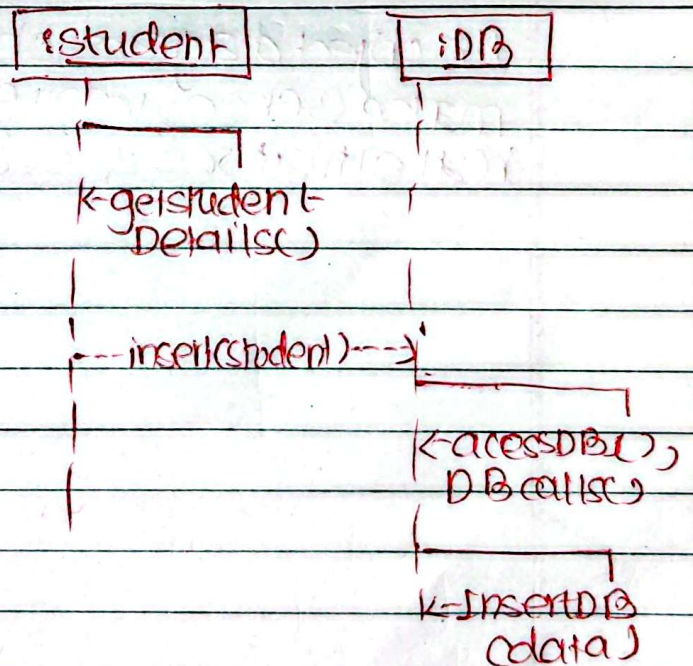
It assign a responsibility so that cohesion remains high

eg:

Low cohesion examples

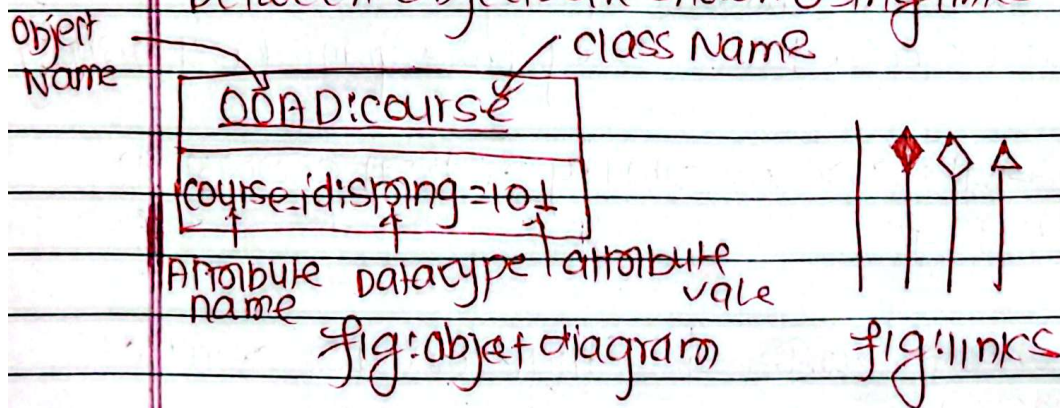


High cohesion example:



k) Object Diagram

→ An object diagram is a UML diagram that represents a snapshot of objects & their relationships at a specific moment in time. Each object in the diagram is represented by object name followed by the class name & the value of its attribute can also be displayed. The relationships between objects are shown using links.



An object diagram checks if the class diagram makes sense, verify that relationship in real situations & see if anything is missing.